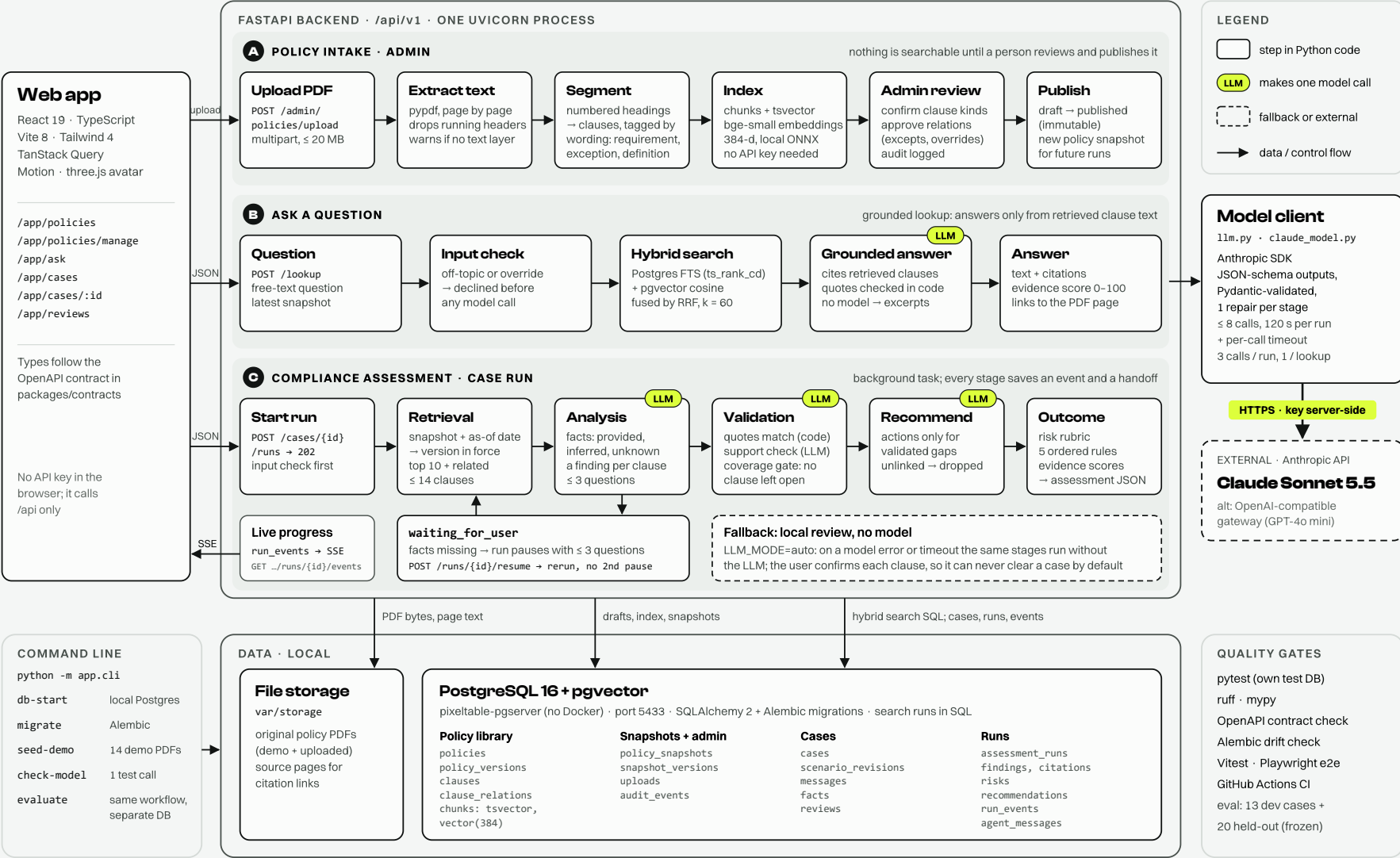
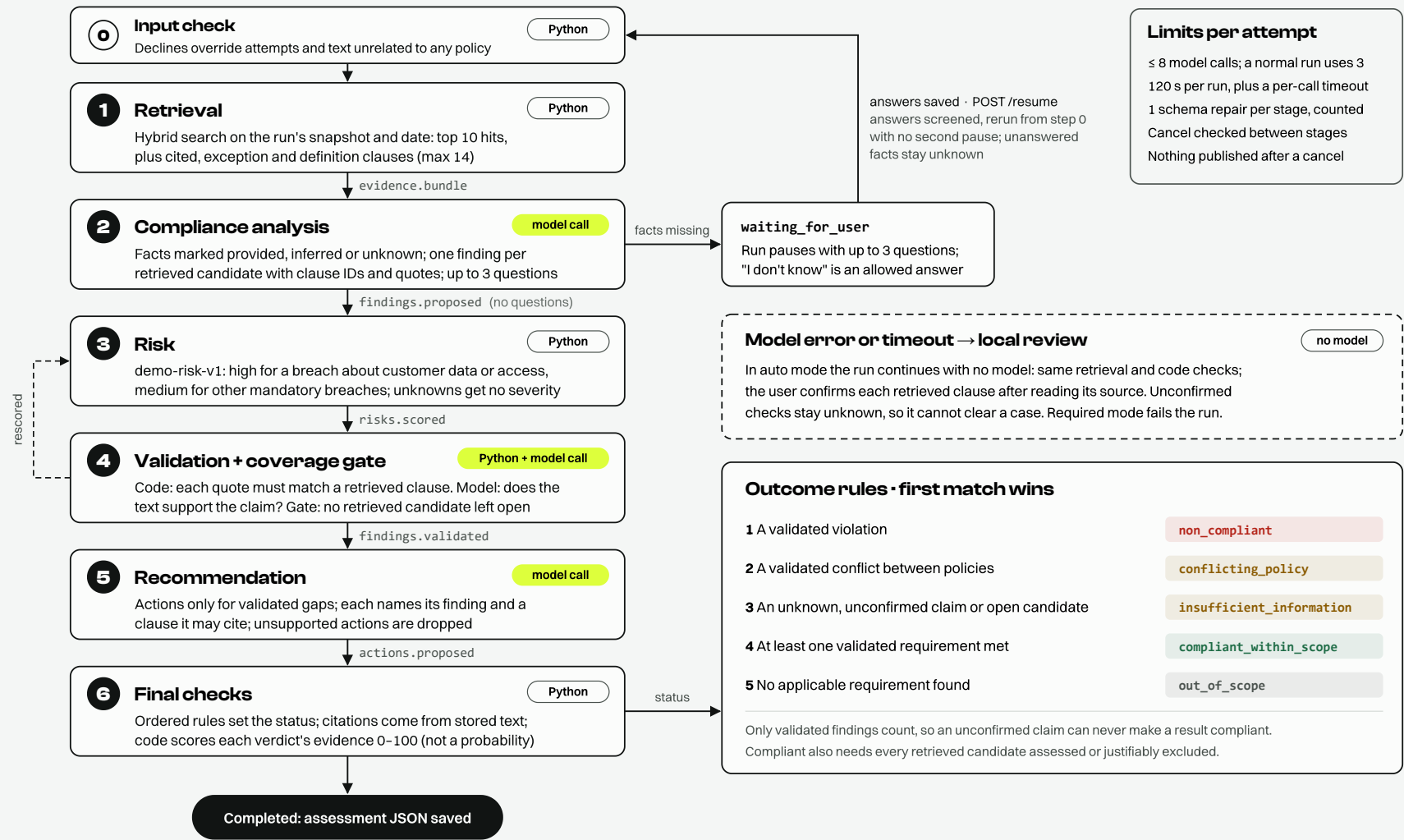




Read it left to right. Three flows share one FastAPI process and one local PostgreSQL database. **A**: an admin uploads a policy PDF; code extracts, segments, tags and indexes it as a draft, a person reviews it, and publishing creates an immutable snapshot that later runs are pinned to. **B**: a question is screened, matched by hybrid search (Postgres full-text + pgvector, fused by reciprocal rank) and answered by one model call whose quotes are verified in code. **C**: a case run retrieves the clauses in force on its as-of date; analysis, validation and recommendation make one model call each, and code decides the status (coverage gate, five ordered rules, evidence scores). Embeddings are computed locally, so the only call leaving the machine is HTTPS to the model provider; if it fails, the run continues as a local review.

Paul.ez — assessment workflow

services/api/app/workflow/ · orchestrator.py runs the stages in order
Five specialist roles and code gates, one backend, one model configuration



Each numbered step is one role with its own function and output schema. After every step the run saves a progress event (streamed to the browser over SSE) and a handoff record naming the sender, receiver and message type shown on the arrows; the trace view reads those records. The input check, retrieval, risk, the coverage gate and the evidence scores are deterministic code; analysis, the validation support check and recommendation are one model call each. The input check runs before any model call and again on the retrieved evidence, so an off-topic request is declined without one. The coverage gate requires a confirmed finding or justified exclusion for every retrieved candidate before a result can be compliant. These are application-level handoffs inside one process, not the standardized Agent2Agent protocol.